Are NBFNet's Path Explanations Actually Faithful? A Deletion, Edit and Neuromodulation Benchmark on a Frozen Knowledge Graph Reasoner

Shadi Khodagholi
University of Central Florida
zahra.khodagholi@ucf.edu

Code and data: https://github.com/shadi97kh/nbfnet-path-faithfulness

Abstract

NBFNet explains a link prediction by reading paths out of the gradient of its own score with respect to each edge, and presents those paths as the provenance of the answer. I test that claim on a frozen public checkpoint by intervening on the graph instead of inspecting it: four graphs, two of them held out, 800 queries the model ranks in its top ten, in a fixed random order, 15,447 single-path deletions, joint deletions, random controls, edits that reroute a final hop, and a dose-response test that turns whole relation types down like a neuromodulator. The audit found a bug. The released explanation code differentiates a version of the model whose messages run backwards, so its gradient is off by as much as the effect it predicts (a median error of 100%) and has the wrong sign on 21 of 50 random directions; a small fix brings the error to 0.7%. With or without the fix, the selected edges are load-bearing: deleting the shortlist removes between 0.259 and 0.858 of the gold tail's probability, against at most 0.007 for as many random edges. The order inside the shortlist carries little: Kendall's τ against measured effect is between -0.009 and 0.076 as released. The fix adds about 0.1, and integrated gradients over the same candidates reach between 0.270 and 0.489, still far from 1 and unchanged with four times the integration steps. Turning a relation down shows why a slope misleads: the response is often a switch, with between 0.030 and 0.290 of a channel's effect present at half gain, where a straight line gives 0.5. The ULTRA paper's own checkpoint reproduces its reported accuracy and gives the same picture. Whether the shortlist is many explanations or one edge written out many times depends on the graph, and the output never says which.

1. Introduction

NBFNet [41] represents a pair of entities by the paths between them, and it explains its own predictions from the same computation. Each edge is given a weight, the model is run, and the gradient of the score with respect to those weights is read off. A beam search over those gradients returns the top paths, and the paper presents them as the paths behind the answer. That the explanation is part of the computation, rather than fitted on top of it, is what makes it attractive and what makes it worth checking.

A gradient tells you what happens if an edge changes a little. Whether the model relied on the edge is a question about taking it away, and the two can disagree. In earlier work on siRNA efficacy models I found saliency maps pointing at sequence positions that mattered far less than the attribution implied, and only perturbing the input showed it [19]. The case for paths over embeddings is provenance: a nearest neighbour in an embedding space such as TransE's [4] cannot tell you why, and a path can. That only works if the path shown is the one doing the work; a reader who checks a wrong path and finds it sound will trust a prediction for the wrong reason.

Where this sits in the course. Topic 8 asked the same question in a different place. Hase and colleagues found that where causal tracing [23] says a fact lives barely predicts where an edit succeeds, with a correlation near -0.13 [14]. MQuAKE found an editor recalling 96.2% of the facts it wrote while multi-hop accuracy on them fell from 40.5% to 7.0% [40], and edits that look reliable collapse once the model generates freely [37]. In each case a method passed an inspection test and failed an intervention test. This project runs that argument on a graph reasoner, where the intervention is literally a knowledge edit and, unlike weight editing, there is no question about whether it took. The model under test is a Topic 2 method, NBFNet inside the ULTRA foundation model, and the research track asks for a course method to be improved and compared one variable at a time. The variables here are the direction of message passing inside the explanation, the attribution rule (gradient or integrated gradients [32]), the checkpoint, and whether the graph was

seen in training. The number of integration steps is the ablation, random deletion is the control, and the reverse-fact case in Section 7.7 is the error analysis, and nDCG [17], the retrieval measure from the course, scores the rankings.

Contributions. (1) A deletion and edit audit of NBFNet's released explanation code, against a frozen public checkpoint, with no training, on a benchmark of four graphs sized by a reliability analysis rather than by budget. (2) A bug in that code: the explanation differentiates the model with every message reversed. NBFNet's own PyTorch Geometric release has the same code pattern, which I read but did not run. I verify it three ways and fix it. (3) The finding that the selected edges are load-bearing on every graph while the order inside the shortlist carries little, before and after the fix, and on the ULTRA paper's own checkpoint. (4) Two measured improvements to that order, the direction fix and integrated gradients [32], neither of which makes it faithful. (5) A relation-level dose response, after Bull *et al.*'s account of neuromodulation [5], that shows the model's channels behave as switches a local slope cannot see. (6) The finding that whether the shortlist is many explanations or one edge depends on the graph, and that the output does not reveal which.

2. Background

NBFNet [41] generalises Bellman-Ford [3] into a neural network. To score $(h, r, ?)$ it initialises the head with a representation of the relation, then runs L rounds of relational message passing [29], so that a node's state summarises the paths of length at most L that reach it from h. I run ULTRA [13], whose entity-level reasoner is NBFNet unchanged, in a class the repository calls `EntityNBFNet`; ULTRA adds a second NBFNet over a graph of relations that conditions the first. That relation-level pass turns the query relation into one vector per relation type, and in the entity graph every message along an edge of type c is multiplied by the vector for c, the DistMult operator [35]. ULTRA ships pre-trained checkpoints in its repository, so the model being explained is a published artefact.

The explanation is not my reimplementation. `BaseNBFNet.visualize` runs the forward pass with a per-edge weight vector marked for automatic differentiation, takes $\partial s / \partial w_e$ for every edge at every layer, beam-searches the highest-scoring paths under those gradients, and returns the top k. I call that function and audit what it returns; Section 5 shows that it differentiates a different model from the one it explains.

3. Related work

Faithfulness. Jacovi and Goldberg argue that faithfulness has to be measured against what the model does, not judged by how plausible an explanation looks [16]. ERASER [10] made two standard measures concrete: comprehensiveness, the drop when the highlighted part is removed, and sufficiency, what survives when only it is kept. I use both, on probabilities so that queries can be compared. Gradient saliency [31] can look right and still fail simple sanity checks [1], and removing what an attribution ranks highest and retraining is a standard test for image models [15].
Graph explanations. GNNExplainer [38] set the pattern of learning a mask that preserves the prediction, followed by PGExplainer [20], SubgraphX [39] and GraphMask [30]. Most of this work produces explanations rather than testing them, and the benchmarks that test them mostly use synthetic graphs with a planted motif [2], which cannot say whether a method matches what a model depends on in a real graph.
Knowledge graphs. For embedding models, CRIAGE [26] and Kelpie [27] explain a prediction by the training facts whose removal changes it, the intervention used here, on models with no paths to read. Rule learners [36], walk-based agents [8] and subgraph reasoners [33] also return paths or rules; NBFNet's appeal is that its paths come from the computation itself, and A*Net [42] scales that design further.
Instability. Models with the same MRR return different top-K predictions across seeds [24], so an explanation from one run deserves error bars. **Neuromodulation.** In nervous systems a few broadcast modulators change what a fixed circuit does without rewiring it [22]; Bull and colleagues model this as a low-dimensional state that multiplies a network's effective connectivity [5]. Such dose responses need not be monotone [7], and the same worry, that a flat local response can hide a large global effect, motivates integrated gradients [32].

4. Method

Write $s_G(h, r, t)$ for the model's score on graph G and $p_G(t \mid h, r)$ for the softmax of that score over all entities. Every comparison is on p, because the logit has no fixed scale across queries, and the gold tail's rank is reported beside it.

Conditions. *A, gradient*: call `visualize` and take its top paths in its order. *B, single-path deletion*: delete each of the top 20 paths from the graph one at a time and measure the drop, which gives the ordering the gradient should have produced. *C, set level*: delete the top 5 jointly (comprehensiveness), and separately keep only their edges (sufficiency). *D, rewire*: point the final hop of the top path, the edge arriving at the predicted tail, at the runner-up entity instead, moving its stored inverse with it, and ask how much the old answer loses and the new one gains.

E, neuromodulatory channels. ULTRA has a factorization like the one Bull *et al.* describe [5]. Its relation-level pass runs once per query and its output multiplies every message of a relation type, so each relation type is a channel that reaches all of its edges at once. I give each channel c a gain g_c that scales every message along relation c and its inverse,

with $g = 1$ the unmodified model. The gradient $\partial p / \partial g_c$ at $g = 1$, one backward pass for all channels, is the local slope: NBFNet's edge-level signal pooled over the channel, taken through the corrected path of Section 5. The dose response is p as g_c is turned down to 0.75, 0.5, 0.25 and 0. It is measured for every channel the top 5 paths use, for the 3 other channels the gradient ranks highest, and, at 0.5 and 0, for 3 random channels as a control; then all explained channels are turned down together. A gain of zero on a set of edges is the same computation as deleting them, so each query first zeroes the gain on exactly the shortlist's edges and checks that this reproduces the deletion recorded in condition C; the largest gap over 550 queries is 0. Turning a channel down is a knowledge edit at the level of a relation rather than a fact.

Interventions are edits. A deletion removes edge columns from `edge_index`, so the model runs on a graph that genuinely lacks those facts. The relation graph that conditions ULTRA is held fixed, so that removing a handful of edges cannot also move the relation representations and mix a second-order change into the measured drop.

Controls. *Random edges*: delete as many uniformly drawn edges as the shortlist contains. *The bottom of the pool*: the method's own lowest-ranked candidates are a within-query null for its ranking. *Tail degree*: a tail with one edge would make any deletion look decisive, so degree is recorded and the central result repeated on tails of degree four or more. *Reverse facts*: some graphs keep symmetric relations, so a query's reverse can already sit in the training graph and make it a one-hop lookup. Each query records whether it does; I added this control after finding the problem in a worked example (Section 7.7).

5. The explanation differentiates a different model

ULTRA, like NBFNet's PyTorch Geometric release, runs its layers with a fused sparse kernel, `rspmm`, whenever it can. When the edge weights need a gradient, which is exactly when `visualize` runs, each layer falls back to PyTorch Geometric's [12] generic message passing. The two disagree on direction. The fused kernel sends the message on column e from `edge_index[1,e]` to `edge_index[0,e]`, and the generic path sends it from `edge_index[0,e]` to `edge_index[1,e]`. Training and inference use the kernel, so that is the deployed model. The explanation differentiates the other one: the same weights with every message running backwards.

I found this because integrated gradients [32] did not add up. Its attributions must sum to the change in p between the empty graph and the full one. With the released gradient they missed it by a median of 63% of that change on the first 30 CoDEx-S queries, against 15% with the corrected one. Three checks on 50 benchmark queries from CoDEx-S and NELL23k then settle it (Figure 1). First, the function the explanation differentiates scores entities up to 10.5 logits away from the deployed model; run on the graph with every edge reversed, the same code reproduces the deployed scores to within 6×10^{-6}. Second, along a random direction in edge-weight space, the corrected gradient predicts the deployed model's change in p to a median of 0.7%, and the released one misses by a median of 100% and has the wrong sign on 21 of the 50 directions. The corrected error is not zero because a finite difference in single precision is itself approximate: with a step of 10^{-3} instead of 10^{-4} it is 0.2%. Third, for each query I take the explained relation channel on which the two gradients disagree most and measure the slope of p against that channel's gain at 1. The released gradient has the wrong sign on 16 of these 50 channels and the corrected one on 3.

The fix is to run `visualize` on the reversed graph and map each hop back to its original edge column. I keep both versions. The released one is what anyone calling the function gets, so the audit below reports it as shipped, and the path benchmark is repeated with the corrected one (Figure 4). The channel gradients and integrated gradients use the corrected path only.

6. The benchmark

Model and graphs. ULTRA's `ultra_4g` checkpoint, 168,705 parameters, frozen, on two CPU cores. It was pretrained on FB15k-237, WN18RR, CoDEx-M and NELL-995. WN18RR [9], from WordNet [25], and FB15k-237 [34] are therefore in its mixture; CoDEx-S [28] and NELL23k [21], from NELL [6], are not, although each comes from the same source as a graph that was. Table 1 gives the graphs and the reproduction, which matters first: if the checkpoint did not reproduce, the thing being explained would not be the published model.

Queries. The test triples are not in the message-passing graph, so an explanation cannot point at the answer itself. They are taken in a fixed random order and kept when the model ranks the gold tail in its top ten, because asking what a prediction rests on only makes sense when the prediction is right. For each query the beam returns up to 20 candidate paths; all are deleted individually, and the top 5 form the shortlist.

How many queries is enough? The model is deterministic: re-running recorded queries from a fresh clone of the code gives every number to the last digit, for every stage. So the only source of variation is which queries are drawn, and every interval below is a 95% bootstrap [11] over queries. To size the sample I computed each headline estimate on the first n queries for growing n, which is a valid sample because the order was fixed in advance (Figure 2). The estimates settle and the bands narrow roughly as $1/\sqrt{n}$. The same analysis shows what a small study would have risked. On WN18RR the disjoint blocks of five queries give a "top

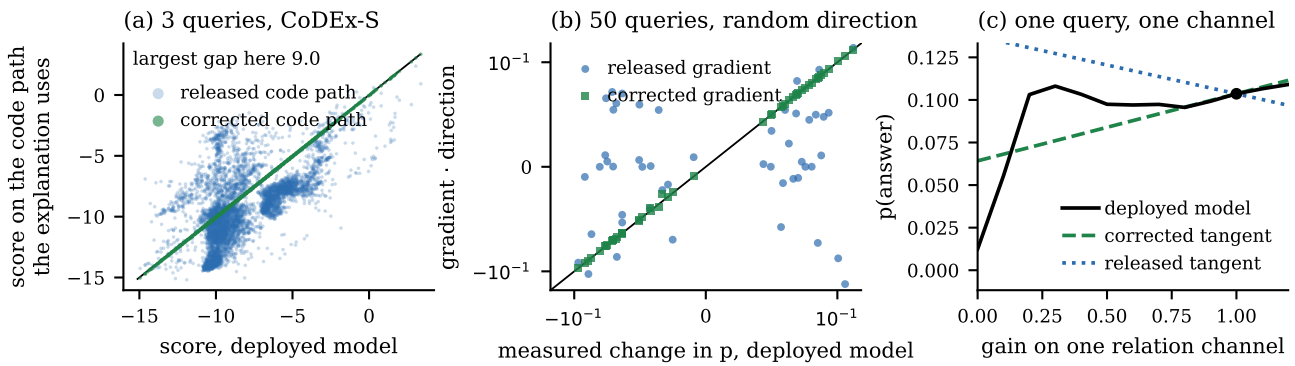


Figure 1. The released explanation code differentiates a model with its messages reversed. (a) Every entity's score from the code path the explanation differentiates, against the deployed model's score: as released (blue) and on the reversed graph (green). (b) Along one random direction in edge-weight space per query, the deployed model's measured change in p, by a central difference, against each gradient's prediction; the line is exact agreement. (c) One query and one relation channel: p as the channel's gain runs from 0 to 1.2, with each gradient's tangent at gain 1.

Table 1. The four graphs. MRR and Hits@k are filtered, both directions scored, with ULTRA's own evaluation code, on the number of test triples shown (filtered ranking [4]; all test triples except on FB15k-237). "Paper" is the MRR Galkin et al. report for ULTRA pre-trained on three graphs [13]. The `ultra_4g` checkpoint used throughout adds NELL-995 to that mixture, so close agreement rather than identity is expected; the "3g" column is that paper's own checkpoint, run here on the full test set. Explained queries are the test triples the frozen model ranks in its top ten, taken in a fixed random order set before any measurement.

graph	entities	relations	edges	`ultra_4g` saw it	test scored	MRR	H@10	paper	3g	queries	scanned
WN18RR	40,943	11	173,670	yes	3134	0.484	0.617	0.480	n/a	200	288
FB15k-237	14,541	237	544,230	yes	2000	0.371	0.558	0.368	n/a	200	400
CoDEx-S	2,034	42	65,776	held out	1828	0.464	0.665	0.472	0.472	200	274
NELL23k	22,925	200	50,890	held out	4952	0.258	0.431	0.239	0.241	200	396

path beats chance" score anywhere from -0.51 to 0.33, and the running estimate still had the opposite sign at 50 queries; at full size it is -0.028, pinned to within ± 0.052. Across random splits of the WN18RR queries into two halves, the two halves' estimates of the shortlist deletion effect differ by at most 0.086 in 95% of splits.

7. Results

7.1. The selected edges are load-bearing

The result that goes the method's way comes first, and it holds everywhere. Deleting the shortlist removes 0.858 of the gold tail's probability on WN18RR, 0.259 on FB15k-237, 0.427 on CoDEx-S and 0.368 on NELL23k (Table 2). Deleting the same number of random edges removes nothing measurable on any graph. The gradient is pointing at part of the graph the model depends on, and that is not a given.

Keeping only the shortlist does not give a clean answer in the other direction. With a handful of edges left, most rival entities are cut off too, so the gold tail can gain probability just because it has fewer competitors. It does on 8.0% of WN18RR queries, 36.5% of FB15k-237, 81.0% of CoDEx-S and 12.0% of NELL23k. I report the retained share for completeness and draw no conclusion about sufficiency from it. For a model that takes a softmax over every entity, a deletion test of necessity is well posed and a keep-only test is not.

7.2. As released, the order is close to meaningless

A faithful ranking would make Kendall's τ [18] between the method's order and the measured drop close to 1. It is -0.009 on WN18RR, 0.048 on FB15k-237, 0.076 on CoDEx-S and 0.041 on NELL23k. Figure 3 shows it directly: walking down the method's own list, the effect of deleting each path stays flat. The plainest version of the question is how often the path ranked first is actually the most damaging one to delete. On WN18RR it is 0.253 against a chance rate of 0.280, no better than a random pick; on the other graphs it is at or modestly above chance. This is the shape of Hase et al.'s result one domain over [14]: a quantity read from the model's internals that correlates with what the model does is not thereby a measurement of what it depends on.

7.3. Fixing the direction helps the order, a little

Repeating the benchmark with the corrected explanation leaves the set-level result where it was (Figure 4): its shortlist

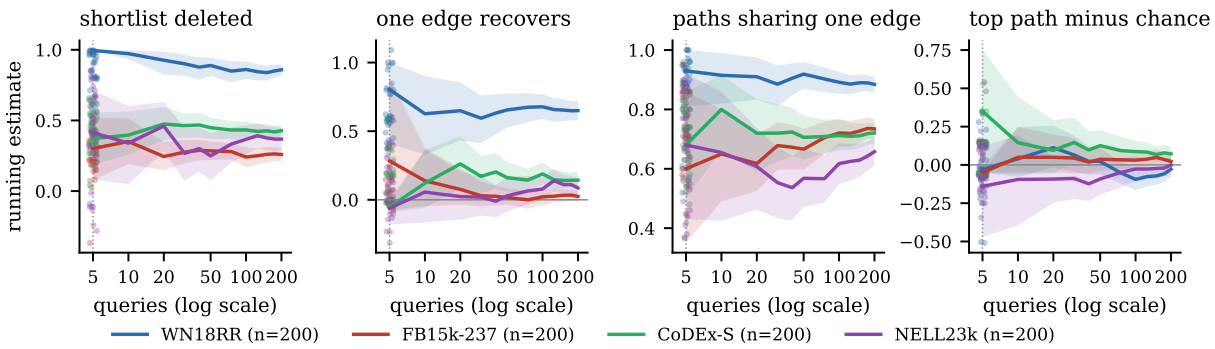


Figure 2. Each headline estimate against the number of queries, as one unbroken line with its 95% bootstrap band, from $n = 5$ to the full sample. The dots at $n = 5$ are every disjoint block of five queries, which is what a five-query benchmark could have reported.

Table 2. Main results. Shares are of each query's own starting probability. Every row uses all 200 queries per graph except the corrected row on FB15k-237 (100), integrated gradients (queries where τ is defined, 200 and 160) and the channel rows (100, 50, 200 and 200). Brackets are 95% bootstrap intervals over queries. "Chance" is the rate at which a random path from the pool would be the most damaging one, counting ties as hits.

	WN18RR	FB15k-237	CoDEx-S	NELL23k
set level: are the selected edges load-bearing?				
delete shortlist, share of p removed	0.858 [0.813, 0.898]	0.259 [0.212, 0.303]	0.427 [0.386, 0.470]	0.368 [0.290, 0.444]
delete as many random edges	0.000	0.000	0.000	0.007
ranking: does the order mean anything?				
Kendall τ, rank against effect	−0.009 [−0.054, 0.037]	0.048 [0.010, 0.085]	0.076 [0.043, 0.108]	0.041 [−0.023, 0.102]
corrected explanation	0.082 [0.036, 0.128]	0.153 [0.098, 0.208]	0.185 [0.146, 0.224]	0.103 [0.042, 0.160]
integrated gradients [32]	0.270 [0.188, 0.353]	0.432 [0.348, 0.513]	0.489 [0.445, 0.531]	0.445 [0.372, 0.516]
top path is the most damaging	0.253	0.080	0.125	0.306
chance	0.280	0.057	0.053	0.307
relation channels, corrected gradient				
silence strongest explained channel	0.995	0.770	0.895	0.895
silence a random channel	0.020	0.000	0.002	0.101
share of effect present at half gain	0.29	0.23	0.19	0.03
gradient has the sign of silencing	59.2%	64.9%	60.8%	73.9%
strongest channel is the query relation	92.0%	85.7%	96.5%	71.5%
redundancy: many explanations or one?				
paths sharing the busiest edge	0.884	0.734	0.721	0.658
distinct edges / edges listed	0.248	0.484	0.512	0.295
that one edge recovers	0.650 [0.581, 0.716]	0.025 [−0.014, 0.066]	0.143 [0.093, 0.197]	0.086 [0.026, 0.147]
structure of the queries				
reverse fact already in graph	57.5%	0.0%	13.0%	4.5%
median tail degree	13	854	320	92

removes 0.461 of the gold tail's probability on CoDEx-S and 0.394 on NELL23k, against 0.427 and 0.368 as released. The order improves. On the queries where τ is defined for both (200 on CoDEx-S, 162 on NELL23k; Table 2 averages over all of them), Kendall's τ rises from 0.076 to 0.185 on CoDEx-S, a gain of 0.109 [0.064, 0.155], and from 0.030 to 0.111 on NELL23k, a gain of 0.081 [0.010, 0.151]; on the seen graphs the gain is 0.082 and 0.076. So the bug explains part of why the order was poor, but not most of it: the right gradient of the right model still ranks paths far from their measured effect; ranking by the measured drops themselves would score 1. ULTRA's three-graph checkpoint, the model of the ULTRA paper, gives the same picture, with τ of 0.182 and 0.109, and it reproduces that paper's MRR, 0.472 against 0.472 on CoDEx-S and 0.241 against 0.239 on NELL23k (Table 1).

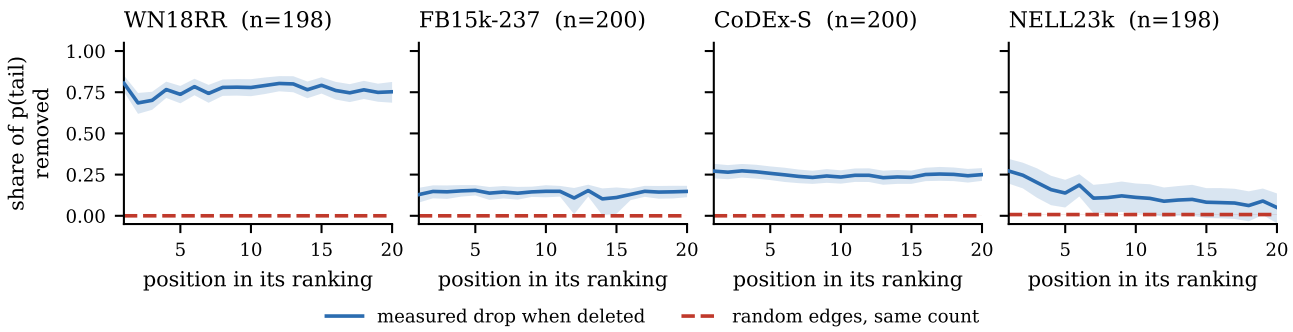


Figure 3. What the method ranks first is not what matters most. The horizontal axis walks down the method's own ordering of its 20 candidates; the vertical axis is the share of the gold tail's probability that deleting that path removes, with a 95% bootstrap band. A faithful ranking would fall from left to right. The dashed line deletes the same number of random edges.

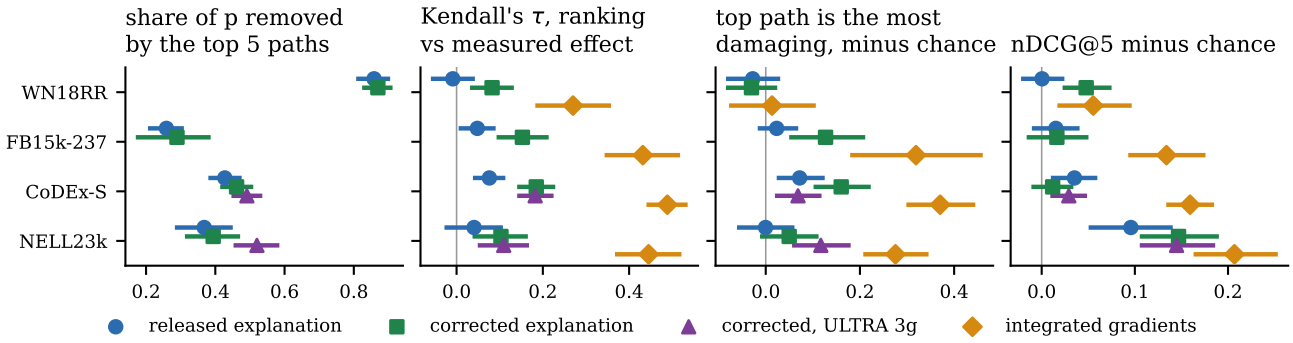


Figure 4. The headline numbers, one variable changed at a time: the released explanation, the direction-corrected one, the corrected one on ULTRA's three-graph checkpoint [13], and integrated gradients [32] re-ranking the corrected explanation's own candidates. Points are means over queries, lines 95% bootstrap intervals. Integrated gradients re-ranks a fixed candidate set, so it has no joint-deletion value. nDCG@5 [17] scores the first five paths with each path's measured drop as its relevance, minus the score of a random order.

7.4. Integrated gradients rank better, but not well

The gradient is a slope at the point where every edge weight is one. Integrated gradients average that slope along the way from the empty graph to the full one [32]. Re-ranking the corrected explanation's own candidates by the integrated gradients summed over each path's edges raises τ on every graph (Figure 4): on the same queries the gain is between 0.179 and 0.305, to between 0.270 and 0.489. On CoDEx-S it goes from 0.185 to 0.489, a paired gain of 0.304 [0.257, 0.350], and how often the top path is the most damaging one, above chance, goes from 0.160 to 0.370. Summed over a relation channel's edges, integrated gradients have the sign of that channel's silencing effect on 90.4% of the channels that matter on CoDEx-S and 86.3% on WN18RR, against 60.8% and 59.7% for the gradient, both averaged per query as in Table 2.

The integral is approximate, so I ablated its one setting. With 16 steps the attributions add up to the total change only to within a median of 15% of it on the first 50 CoDEx-S queries; with 32 and 64 steps the error falls to 11% and 5%, while τ stays at 0.467, 0.472 and 0.456, and the 16-step and 64-step orders agree with $\tau = 0.903$. The gain is not an artifact of a coarse integral. It costs 16 backward passes instead of one, and it is the largest improvement in the study, yet τ stays at or below 0.5: the order of the paths is only loosely tied to what deleting them does.

7.5. Relation channels are switches, not dials

Turning a channel down shows why a slope misleads (Figure 5). Silencing the strongest channel the explanation uses removes 0.895 of the gold tail's probability on CoDEx-S and 0.895 on NELL23k, while a random channel removes 0.002 and 0.101 (Table 2). The response is not a dial. For channels whose silencing matters, only 0.19 of the full effect is present at half gain on CoDEx-S, where a straight line would give 0.5. 51.7% of them have less than a quarter of it, and 35.6% change direction somewhere along the dose, a non-monotone response of the kind reported for dopamine [7]. Turning every explained channel down to half together removes nothing measurable on CoDEx-S (-0.027 [-0.071, 0.017]) and raises p

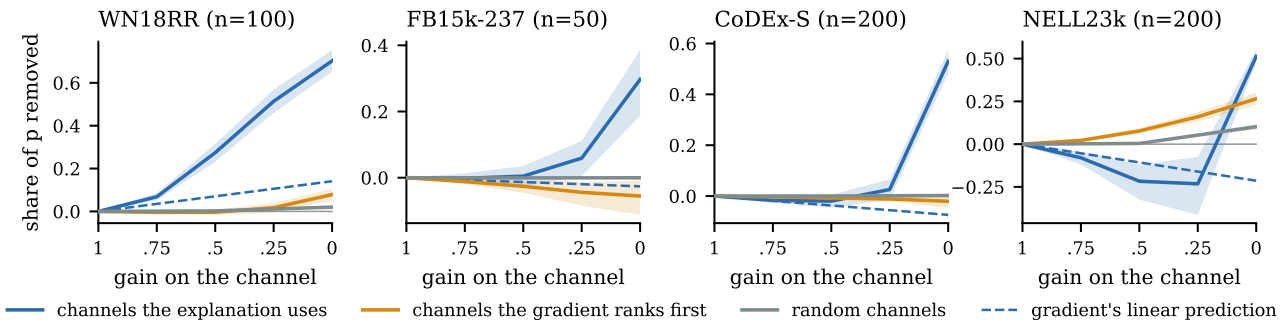


Figure 5. Turning one relation channel down from gain 1 to 0. Curves are the share of the gold tail's probability removed, averaged over queries, with 95% bootstrap bands, for the channels the explanation's paths use, the other channels the corrected gradient ranks highest, and random channels. The dashed line is the gradient's linear prediction for the explained channels.

on NELL23k (−0.181), against 0.909 at zero on CoDEx-S. For the explained channels turned down together, WN18RR is the exception: half gain already removes 0.489, about what a straight line gives. A slope taken at gain 1 cannot see a switch that sits near 0, and the corrected gradient has the sign of the silencing effect for only 60.8% of these channels on CoDEx-S and 73.9% on NELL23k, where chance is about half. The channel that matters most is usually the query's own relation, from 71.5% of queries on NELL23k to 96.5% on CoDEx-S. The paths do name that relation, as a hop; what they cannot show is that it acts as a switch. This is the Topic 8 result again, one level up: the gradient localises a channel, and localising it does not predict what editing it does.

7.6. One edge or many depends on the graph

The shortlist is presented as several independent paths. On WN18RR it is close to one. A single edge runs through 0.884 of the listed paths, only 0.248 of the listed edge slots are distinct, and deleting that one edge reproduces 0.650 of the effect of deleting the whole shortlist. The obvious objection is that these tails are poorly connected so that any deletion strands them; it fails, since the median tail degree is 13 and the result holds on tails of degree four or more (0.613).

The other graphs do not look like this. On FB15k-237 the busiest edge appears in 0.734 of paths and deleting it recovers 0.025 of the effect, an interval that includes zero; the paths there are genuinely different routes. The structural difference is in the graphs themselves (Tables 1 and 2): WN18RR is sparse, has 11 relation types and keeps symmetric relations, so 57.5% of its queries have their reverse fact in the graph, while FB15k-237 has 237 relation types, a median tail degree of 854, and 0.0% of queries with a reverse fact. The held-out graphs sit between the two and let the pattern be checked on graphs the model never saw.

Table 3. WN18RR split on whether the query's reverse fact is in the training graph. The forward edge is never present (0.0%).

	reverse present (115 queries)	reverse absent (85 queries)
paths sharing one edge	0.894	0.872
Kendall τ	−0.012	−0.003
top path is best	0.157	0.392
chance	0.217	0.372
one edge recovers	0.807	0.395
rewire, old loses	0.2808	0.0017

7.7. The control I found by reading an example

While building a worked example on WN18RR, the top "path" came back as a single hop, and that hop was the stored reverse of a training fact. Table 4 shows the first such query in the benchmark's order. The model is asked what *sociology* is derivationally related to and answers *sociologist*, because the training graph already holds that fact the other way round. The explanation offers five paths. Two of them detour from sociology to *mores* and straight back before taking the same reverse hop, and deleting that one hop alone does everything that deleting all five paths does. This is not leakage, since the test edge itself is never in the graph, but it changes what there is to explain, so Table 3 splits WN18RR on it. Two findings survive the split: the paths funnel through one edge on both halves, and the ranking is no better than chance on both. What does not survive is how much that edge carries, 0.807 with the reverse fact present against 0.395 without, and almost all of the rewire effect. The first column is the sharpest thing in this paper. On those queries the model is doing a one-hop lookup, and the interface returns up to 20 paths of up to six hops. Nothing in that output says the answer came from one reverse lookup, which is the one thing a reader would most want to know.

Table 4. A worked example, the first query in the benchmark's fixed order that meets the rule in its heading, re-run under the benchmark protocol. Arrows show the direction of the stored fact, so a leftward arrow is a hop taken backwards along a reverse edge. Names are from the WN18RR text release.

WN18RR, first query with its reverse fact in the graph. Query: sociology − derivationally related form → ? answer **sociologist**, rank 1, $p = 0.196$	
path 1 (1 hop)	sociology ← derivationally related form − sociologist
path 2 (3 hops)	sociology ← synset domain topic of − mores
	mores − synset domain topic of → sociology
	sociology ← derivationally related form − sociologist
paths 3 to 4	longer loops that end in the same last hop as path 1
delete top 5 (6 edges)	p 0.196 → 0.000, rank 1 → 24859
delete busiest edge only	p 0.196 → 0.000, rank 1 → 24874: 100% of that effect
	that edge, in 4 of 5 paths: sociologist − derivationally related form → sociology

7.8. Rerouting a path destroys rather than redirects

If the top path were the mechanism, pointing its last hop at another entity should carry the answer there. On WN18RR the old answer loses 0.1622 while the new target gains 0.0282, almost all of it on reverse-fact queries; elsewhere neither moves much. The edit takes support away without moving it, MQuAKE's portability failure [40] with an exact edit.

8. Discussion

The bug is the most concrete finding, and its lesson is general. A gradient explanation is a statement about whatever code was differentiated, and here that was not the code that makes the predictions. Nothing in the output looks wrong, and the deletion tests did not catch it. Checking the gradient against the model's own response did: the completeness of integrated gradients [32], then the scores and a finite difference.

The gradient finds a region of the graph the model depends on, and random deletion confirms it is not luck. As released it ranks little or nothing inside that region, and even fixed it ranks poorly; whether it decomposes the region into separate evidence depends on the graph. A heat map over edges and a ranked list of reasoning chains are different promises. NBFNet's gradient keeps the first and is presented as the second.

Ten paths that agree feel like ten witnesses. If nine are the same witness in different coats, a reader who spot-checks two has learned far less than they think, and the output looks the same either way. Five changes would help: run the explanation on the reversed graph; rank with integrated gradients [32], at 16 backward passes, if an order is going to be shown; report the busiest edge and confirm it with one deletion; print the distinct-edge count beside any list of paths; and flag queries whose reverse fact is already in the graph.

None of this says the model is broken. Its accuracy is in line with what the ULTRA paper reports, and the edges its gradient selects are load-bearing. What fails is the explanation layer: first a gradient taken through the wrong code path, and then, once that is fixed, a real but coarse signal over edges, served as a ranked list of discrete reasoning chains it does not support. The channel experiment suggests why a better gradient is not enough: much of the answer rides on relation channels that act as switches, which a slope cannot see.

9. Limitations

Overlapping paths. Deleting a path disturbs other paths that share its edges, so a single-path drop is not cleanly that path's. The redundancy rows of Table 2 measure this, and the set-level and single-edge numbers do not depend on it. **One model family**, four graphs: the claim is about this family's explanations, not path reasoning in general. **Correct predictions only**: what the explanation does on the model's mistakes is a separate question. **Smaller runs on the seen graphs**: the corrected benchmark has 100 queries on FB15k-237, the channel experiment 100 on WN18RR and 50 on FB15k-237, and integrated gradients 86 and 50, against 200 elsewhere; the second checkpoint was run on the held-out graphs only. **NBFNet-PyG**: its layer code has the same fallback, but I ran the checks on ULTRA only. **WN18RR node merges**: the release PyTorch Geometric loads merges 161 unrelated synsets (0.4%) into shared nodes; the checkpoint was trained on that graph, so I left it.

10. Conclusion

I tested whether the paths NBFNet reports are the paths it relies on, by editing a frozen checkpoint's graph rather than reading its gradients, on 800 queries over four graphs. The released explanation differentiates the model with its messages reversed; fixed, it matches the deployed model. Either way the selected edges are load-bearing, and the order they come in carries little: the fix and integrated gradients each improve it, and neither makes it faithful. Localising is not explaining, and only changing the graph tells the two apart.

References

[1] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. 2

[2] Chirag Agarwal, Owen Queen, Himabindu Lakkaraju, and Marinka Zitnik. Evaluating explainability for graph neural networks. *Scientific Data*, 10, 2023. 2

[3] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958. 2

[4] Antoine Bordes, Nicolas Usunier, Alberto García-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2787–2795, 2013. 1, 4

[5] Matthew S. Bull, Po-Chen Kuo, Andrew L. Smith, and Michael A. Buice. Volume transmission implements context factorization to target online credit assignment and enable compositional generalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2

[6] Andrew Carlson, Justin Betteridge, Bryan Kisiel, Burr Settles, Estevam R. Hruschka Jr., and Tom M. Mitchell. Toward an architecture for never-ending language learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2010. 3

[7] Roshan Cools and Mark D'Esposito. Inverted-U-shaped dopamine actions on human working memory and cognitive control. *Biological Psychiatry*, 69(12):e113–e125, 2011. 2, 6

[8] Rajarshi Das, Shehzaad Dhuliawala, Manzil Zaheer, Luke Vilnis, Ishan Durugkar, Akshay Krishnamurthy, Alex Smola, and Andrew McCallum. Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2018. 2

[9] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. Convolutional 2D knowledge graph embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. arXiv:1707.01476; source of the FB15k-237 and WN18RR splits. 3

[10] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C. Wallace. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4443–4458, 2020. arXiv:1911.03429. 2

[11] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. 3

[12] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. 3

[13] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.04562. 2, 4, 6

[14] Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2301.04213. 1, 4

[15] Sara Hooker, Dumitru Erhan, Pieter-Jan Kindermans, and Been Kim. A benchmark for interpretability methods in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 2

[16] Alon Jacovi and Yoav Goldberg. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4198–4205, 2020. 2

[17] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. 2, 6

[18] Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938. 4

[19] Zahra Khodagholi and Niloofar Yousefi. Validating interpretability in siRNA efficacy prediction: A perturbation-based, dataset-aware protocol. In *MLGenX Workshop, International Conference on Learning Representations (ICLR)*, 2026. arXiv:2602.10152. 1

[20] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. Parameterized explainer for graph neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2

[21] Xin Lv, Xu Han, Lei Hou, Juanzi Li, Zhiyuan Liu, Wei Zhang, Yichi Zhang, Hao Kong, and Suhui Wu. Dynamic anticipation and completion for multi-hop reasoning over sparse knowledge graph. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5694–5703, 2020. 3

[22] Eve Marder. Neuromodulation of neuronal circuits: Back to the future. *Neuron*, 76(1):1–11, 2012. 2

[23] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1

[24] Guillaume Méroué, Fabien Gandon, and Pierre Monnin. Link prediction or perdition: the seeds of instability in knowledge graph embeddings. *arXiv preprint*, 2026. arXiv:2606.03365. 2

[25] George A. Miller. WordNet: A lexical database for English. *Communications of the ACM*, 38(11):39–41, 1995. 3

[26] Pouya Pezeshkpour, Yifan Tian, and Sameer Singh. Investigating robustness and interpretability of link prediction via adversarial modifications. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 3336–3347, 2019. 2

[27] Andrea Rossi, Donatella Firmani, Paolo Merialdo, and Tommaso Teofili. Explaining link prediction systems based on knowledge graph embeddings. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 2062–2075, 2022. 2

[28] Tara Safavi and Danai Koutra. CoDEx: A comprehensive knowledge graph completion benchmark. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8328–8350, 2020. 3

[29] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web: European Semantic Web Conference (ESWC)*, pages 593–607, 2018. 2

[30] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. Interpreting graph neural networks for NLP with differentiable edge masking. In *International Conference on Learning Representations (ICLR)*, 2021. 2

[31] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014. 2

[32] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning (ICML)*, pages 3319–3328, 2017. 1, 2, 3, 5, 6, 8

[33] Komal K. Teru, Etienne G. Denis, and William L. Hamilton. Inductive relation prediction by subgraph reasoning. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 9448–9457, 2020. 2

[34] Kristina Toutanova and Danqi Chen. Observed versus latent features for knowledge base and text inference. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality*, pages 57–66, 2015. 3

[35] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations (ICLR)*, 2015. 2

[36] Fan Yang, Zhilin Yang, and William W. Cohen. Differentiable learning of logical rules for knowledge base reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2

[37] Wanli Yang, Fei Sun, Jiajun Tan, Xinyu Ma, Du Su, Dawei Yin, and Huawei Shen. The mirage of model editing: Revisiting evaluation in the WILD. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2502.11177. 1

[38] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. GNNExplainer: Generating explanations for graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1903.03894. 2

[39] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. On explainability of graph neural networks via subgraph explorations. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 12241–12252, 2021. 2

[40] Zexuan Zhong, Zhengxuan Wu, Christopher D. Manning, Christopher Potts, and Danqi Chen. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.14795. 1, 8

[41] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2106.06935. 1, 2

[42] Zhaocheng Zhu, Xinyu Yuan, Mikhail Galkin, Sophie Xhonneux, Ming Zhang, Maxime Gazeau, and Jian Tang. A*Net: A scalable path-based reasoning approach for knowledge graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2206.04798. 2